Les Modèles Algorithmiques Fondamentaux sur Structures à Accès Direct

Thème principal : Modèles algorithmiques canoniques, mécanismes d'accès aux structures de données et gestion des bornes

Synthèse exécutive : Ce cours établit les fondements de la conception algorithmique rigoureuse en BUT 1 Informatique. Avant toute écriture de code, un informaticien doit analyser le mécanisme d'accès de sa structure de données (Direct, Séquentiel ou Dynamique). Le cours détaille les trois modèles canoniques sur structures à accès direct (tableaux, chaînes de caractères) : le parcours complet systématique, le parcours complet conditionné, et la recherche de première occurrence. Une attention particulière est portée à l'évitement des erreurs d'indices (out of bounds) et aux exigences strictes de la recherche (pessimisme algorithmique et double condition d'arrêt obligatoire).

1. Définitions & Typologie des Mécanismes d'Accès aux Données

L'analyse préalable du mécanisme d'accès est l'étape zéro de toute conception algorithmique. Si une structure possède un accès direct (comme un tableau de N éléments ou une chaîne de caractères), tout élément situé à la position 'i' est immédiatement accessible par l'instruction T[i]. À l'inverse, sur une structure à accès séquentiel (comme un fichier texte brut, un flux CSV ou une liste chaînée simple), il est impossible d'accéder directement à T[50] : il faut exécuter 50 fois l'opération 'suivant' à partir du début. Tenter d'appliquer un algorithme basé sur l'accès direct (comme une boucle Pour i de 0 à N-1 avec accès T[i]) sur un flux séquentiel constitue une erreur de conception fondamentale qui invalide totalement l'algorithme.

- Structure de données : Collection organisée regroupant des éléments de même type (ex: tableaux, chaînes de caractères).
- Accès Direct : Capacité d'atteindre instantanément n'importe quel élément via son indice ou sa clé, sans avoir à parcourir les éléments précédents.
- Accès Séquentiel : Contrainte matérielle ou logique imposant de démarrer au début de la structure et de progresser pas à pas (début, suivant, fin).
- Accès Dynamique : Structure hybride offrant à la fois des primitives d'accès séquentiel et d'accès direct.

Cas concret enseignant : *Le professeur insiste sur un piège classique en évaluation : utiliser une notation d'accès direct T[i] sur un fichier CSV brut ou un fichier texte. Ces flux étant gérés séquentiellement ligne par ligne par le système, une telle opération est physiquement impossible et entraîne une note zéro sur la question.*

2. Modèle 1 : Parcours Complet avec Traitement Systématique

Ce modèle s'applique lorsque chaque élément d'une collection doit subit un traitement uniforme et inconditionnel (ex: initialiser un tableau, afficher tous ses éléments, passer tous les caractères d'une chaîne en majuscules). Comme le nombre d'éléments à traiter est fixe et déterminé dès le départ (Nombre = Fin - Début + 1), la structure itérative obligatoire est la boucle bornée 'Pour'. Schéma algorithmique général : Pour i allant de BorneDébut à BorneFin Faire Traiter(Collection[i]) Fin Pour Sur un tableau de N éléments en notation standard (indexé de 0 à N-1), la boucle s'écrit 'Pour i allant de 0 à N - 1'. L'erreur classique est de faire varier 'i' jusqu'à N, ce qui tente d'accéder à la case N (inexistante) et déclenche une faute fatale de dépassement d'indice (out of bounds).

- Exhaustivité : 100% des éléments de la zone définie (entre borne début et borne fin) subissent le traitement.
- Structure itérative : Boucle bornée 'Pour' (nombre d'itérations connu avant le démarrage).
- Gestion des bornes : Pour une collection de N éléments indicée à partir de 0, l'indice varie strictement de 0 à N - 1.

Cas concret enseignant : *Exemple d'illustration du prof : Passer en majuscules tous les caractères d'un mot de passe ou afficher toutes les cases d'un tableau d'entiers. Le traitement est appliqué aveuglément à chaque itération.*

3. Modèle 2 : Parcours Complet avec Traitement Conditionné

Le parcours complet conditionné conserve l'exploration exhaustive du modèle 1, mais protège l'action de traitement par une condition logique. Il est utilisé pour compter, afficher ou transformer uniquement une sous-population d'éléments répondant à un critère (ex: compter les voyelles dans une chaîne, afficher les nombres pairs d'un tableau). Schéma algorithmique général : Pour i allant de BorneDébut à BorneFin Faire Si Propriété(Collection[i]) Alors Traiter(Collection[i]) Fin Si Fin Pour La condition peut être simple (ex: T[i] > 0), complexe (ex: T[i] > 0 ET T[i] mod 2 = 0), ou même impliquer des éléments voisins (ex: comparer T[i] avec T[i-1]), sous réserve d'ajuster correctement les bornes pour éviter tout débordement mémoire.

- Inspection complète : Tous les éléments de la zone sont visités de la borne initiale à la borne finale.
- Filtrage logique : Le traitement n'est exécuté que si l'élément courant valide une condition booléenne.
- Combinaison de structures : Imbrication d'une structure alternative 'Si ... Alors' à l'intérieur d'une boucle bornée 'Pour'.

Cas concret enseignant : *L'analogie de la chaîne de montage : Dans une usine de contrôle qualité, chaque pièce défile devant l'inspecteur (parcours complet de la chaîne), mais l'inspecteur ne met un coup de tampon de rebut (traitement conditionné) que si la pièce présente un défaut spécifique.*

4. Modèle 3 : Recherche de Première Occurrence

Contrairement aux parcours complets, la recherche de première occurrence s'appuie sur une itération non bornée ('Tant que'), car le nombre de répétitions est inconnu à l'avance. Un algorithme de recherche valide repose sur le principe du pessimisme : on suppose au départ que l'élément n'existe pas (trouve <- Faux). Exigence fondamentale : Il doit comporter DEUX conditions d'arrêt : 1. Succès : L'élément recherché a été trouvé (trouve = Vrai). 2. Échec : Tous les éléments ont été explorés sans succès (Fin de structure atteinte). Variante 1 (Rupture) : Dès que l'élément est trouvé, 'trouve' passe à Vrai et l'algorithme interrompt la boucle. L'indice 'i' pointe alors exactement sur la position de l'élément trouvé. Variante 2 (Test unique en tête) : La boucle est conditionnée par 'i < N et NON trouve'. Si l'élément est trouvé, 'trouve' devient Vrai, mais la boucle effectue une dernière incrémentation de 'i' avant de réévaluer le test, nécessitant un réajustement d'indice s'il faut exploiter la position exacte.

- Objectif : Localiser la tout premier élément vérifiant une propriété et interrompre immédiatement l'exploration.
- Pessimisme algorithmique : Initialisation systématique d'un indicateur booléen 'trouvé' à Faux avant d'entrer dans la boucle.
- Double condition d'arrêt obligatoire : La recherche doit s'arrêter SI l'élément est trouvé OU SI la structure est entièrement parcourue.
- Variante 1 (Rupture immédiate) : S'arrête exactement à l'indice où l'élément a été trouvé.
- Variante 2 (Tant que global) : Regroupe la double condition en tête de boucle ('Tant que i < N et NON trouvé').

Cas concret enseignant : *Devise du prof : 'En informatique, soyez pessimistes ! Partez du principe que ce que vous cherchez n'existe pas, et ne changez votre indicateur que lorsque vous le voyez.' Rédiger une recherche avec une seule condition d'arrêt est une erreur majeure sanctionnée en partiel.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui, parfaitement ! Une structure à accès dynamique offre à la fois des primitives d'accès séquentiel et d'accès direct. On peut donc tout à fait utiliser ses mécanismes d'accès direct pour y appliquer un parcours complet systématique.

Question étudiant : Pourquoi présenter deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : Parce qu'elles répondent à des besoins complémentaires : La Variante 1 est plus précise si l'on a besoin d'isoler l'indice exact de l'élément au moment de l'arrêt. La Variante 2 regroupe la double condition d'arrêt en tête de boucle ('Tant que'), ce qui est plus élégant et standard sur le plan syntaxique, mais demande un léger réajustement de l'indice si l'on cherche sa position.

Question étudiant : Comment fait-on si la condition de recherche dans un parcours conditionnel dépend de l'élément précédent ou suivant ?
Réponse du professeur : Le modèle reste le même (parcours conditionné), mais l'adaptation nécessite une vigilance accrue sur les bornes. Si le test compare T[i] à T[i-1], la boucle doit impérativement commencer à i = 1 (et non 0) pour ne pas sortir du tableau au premier tour.

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur donnée (ex: trouver tous les '5' d'un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Ce n'est plus une recherche de première occurrence (qui s'arrête dès qu'elle trouve une occurrence). C'est un parcours complet avec traitement conditionné, où l'on visite toutes les cases et où le traitement consiste à stocker ou afficher l'indice chaque fois que la valeur est égale à 5.

■ Exercice & Démonstration d'Application

Énoncé : Écrire l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments, en utilisant le modèle algorithmique approprié.

Résolution pas à pas :
1. Analyse du problème : Il faut examiner la totalité des éléments d'un tableau (structure à accès direct) de la première à la dernière case. L'affichage (traitement) ne doit s'exécuter que si la valeur est strictement supérieure à 0. Le modèle adapté est le 'Parcours complet avec traitement conditionné'. 2. Détermination des bornes : Le tableau comprend N cases indicées de 0 à N - 1. Borne de début = 0, Borne de fin = N - 1. 3. Choix de la boucle : Une boucle bornée 'Pour i allant de 0 à N - 1'. 4. Condition de filtrage : À chaque itération, tester 'Si T[i] > 0 Alors'. 5. Action : Exécuter 'Afficher(T[i])'.

Résultat : Algorithme AffichagePositifs Données : T : Tableau d'entiers, N : Entier Variables : i : Entier Début Pour i allant de 0 à N - 1 Faire Si T[i] > 0 Alors Afficher(T[i]) Fin Si Fin Pour Fin Commentaire : L'algorithme garantit un parcours exhaustif sans risque de débordement d'indice (bornes 0 à N-1) et filtre exactement les valeurs strictement positives.

■ Auto-Évaluation Flash d'Examen

Q : Quel modèle algorithmique doit-on utiliser pour vérifier si un tableau d'entiers contient au moins une fois la valeur 10 ?

R : La Recherche de Première Occurrence, car l'objectif est de vérifier une présence et de s'interrompre dès la première découverte.

Q : Pourquoi l'indice de fin d'un parcours complet sur un tableau de N éléments est-il égal à N - 1 et non à N ?

R : Parce que les indices d'un tableau de N cases sont numérotés de 0 à N - 1. Accéder à l'indice N provoquerait une erreur de débordement mémoire (out of bounds).

Q : Combien de conditions d'arrêt minimales doit comporter la structure itérative d'une recherche de première occurrence ?

R : Au moins deux conditions d'arrêt obligatoires : (1) L'élément recherché a été trouvé, et (2) La totalité de la structure a été parcourue (échéance/fin de collection).